

Mastering Time Travel in Snowflake: Query the Past Like a Pro

Data mistakes happen.

Someone accidentally deletes records.

A bad deployment overwrites data.

An ETL job corrupts a table at 2 AM.

In traditional databases, recovery can be painful and sometimes impossible.

But in Snowflake, there's a superpower called **Time Travel** that allows you to access historical data from the past — almost like having a built-in undo button for your database.

In this article, we'll explore:

- What Time Travel is
 - What you can and cannot do with it
 - How it works internally
 - Retention periods
 - Querying historical data
 - Restoring deleted tables
 - Real-world examples
 - Best practices
-

What is Time Travel?

Time Travel in Snowflake allows you to:

- Access historical versions of data
- Query data from the past
- Recover deleted objects
- Restore accidentally modified data

It works for:

- Tables
- Schemas
- Databases

Snowflake stores previous versions of data for a configurable retention period.

What You CAN Do with Time Travel

Time Travel is extremely powerful for recovery, auditing, and debugging.

1. Query Historical Data

```
SELECT *  
FROM employees  
AT (OFFSET => -3600);
```

This queries the table from 1 hour ago.

2. Recover Deleted Tables

```
DROP TABLE employees;
```

Recover it instantly:

```
UNDROP TABLE employees;
```

3. Recover Deleted Schemas and Databases

```
UNDROP SCHEMA sales_schema;
```

```
UNDROP DATABASE sales_db;
```

4. Compare Old vs Current Data

```
SELECT  
    curr.id,  
    curr.salary AS current_salary,  
    old.salary AS old_salary  
FROM employees curr  
JOIN employees AT (OFFSET => -3600) old  
ON curr.id = old.id;
```

5. Clone Historical Data

```
CREATE TABLE employees_backup  
CLONE employees  
AT (OFFSET => -7200);
```

6. Recover Accidentally Deleted Rows

```
DELETE FROM orders WHERE status = 'ACTIVE';
```

Restore using historical snapshot:

```
INSERT INTO orders  
SELECT *  
FROM orders  
AT (OFFSET => -300)  
WHERE status = 'ACTIVE';
```

7. Audit Historical State

Useful for:

- Auditing
 - Compliance
 - Root cause analysis
 - Financial reporting
-

8. Recover from Bad Updates or Merges

Retrieve previous versions after bad updates or merges.

What You CANNOT Do with Time Travel

1. You Cannot Recover Data After Retention Expires

Once retention ends, historical data is permanently unavailable.

2. You Cannot Time Travel Forever

Maximum retention:

- Standard Edition → 1 day
 - Enterprise Edition → up to 90 days
-

3. You Cannot Recover Data After Fail-safe Ends

Fail-safe is not directly accessible to users.

4. You Cannot Use Time Travel on External Tables Like Regular Tables

External tables have limited support.

5. You Cannot Restore Individual Columns

Time Travel restores object/data versions, not individual columns.

6. You Cannot Use Time Travel if the Object Did Not Exist

Historical queries fail if the object didn't exist at that time.

7. You Cannot Recover Temporary Tables After Session Ends

Temporary tables disappear after session termination.

8. You Cannot Avoid Storage Costs

Historical versions consume storage.

9. You Cannot Recover Privileges Automatically in Every Scenario

Always verify grants after recovery.

10. You Cannot Use Time Travel as a Backup Replacement

It complements backups — it does not replace them.

Time Travel Retention Period

Edition	Retention Period
Standard	1 day
Enterprise	Up to 90 days

Example:

```
ALTER TABLE employees
SET DATA_RETENTION_TIME_IN_DAYS = 7;
```

Creating Sample Data

```
CREATE OR REPLACE TABLE employees (
  id INT,
  name STRING,
  salary NUMBER
);
```

```
INSERT INTO employees VALUES
(1, 'Alice', 50000),
(2, 'Bob', 60000),
(3, 'Charlie', 70000);
```

Query Historical Data Using AT

Using OFFSET

```
SELECT *
FROM employees
AT (OFFSET => -3600);
```

Using TIMESTAMP

```
SELECT *
FROM employees
AT (
  TIMESTAMP => '2026-05-11 10:00:00'::TIMESTAMP
);
```

Using STATEMENT

```
SELECT *  
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())  
ORDER BY START_TIME DESC;
```

Then:

```
SELECT *  
FROM employees  
BEFORE(STATEMENT => '01b12345-0001-aaaa-0000-123456789abc');
```

Understanding How Snowflake Stores Historical Data

Snowflake uses:

- Immutable micro-partitions
- Metadata tracking
- Copy-on-write architecture

Instead of physically rewriting files, Snowflake maintains references to historical versions.

Time Travel vs Fail-safe

Feature	Time Travel	Fail-safe
User Accessible	Yes	No
Query Historical Data	Yes	No
Recover Dropped Objects	Yes	Limited
Used For	Operational recovery	Disaster recovery
Retention	Up to 90 days	Additional 7 days
Recoverable By	User	Snowflake Support

Best Practices

Keep Retention Based on Business Needs

Longer retention = higher storage costs.

Use Cloning Before Risky Operations

```
CREATE TABLE employees_clone CLONE employees;
```

Monitor Storage Usage

Historical data contributes to storage billing.

Use Statement-Based Recovery

Combining query history with Time Travel is very powerful.

Final Thoughts

Time Travel is one of Snowflake's most valuable features.

It gives your data warehouse memory.

With it, you can:

- Recover deleted data
- Debug production issues
- Audit historical states
- Restore corrupted datasets
- Safely experiment with transformations

But understanding its limitations is equally important.

Quick Cheat Sheet

Task	Command
Query old data	AT (OFFSET => -3600)
Query exact time	AT (TIMESTAMP => ...)
Query before statement	BEFORE(STATEMENT => ...)
Restore dropped table	UNDROP TABLE table_name
Clone old version	CLONE ... AT (...)
Set retention	ALTER TABLE ... SET DATA_RETENTION_TIME_IN_DAYS

Task	Command
------	---------

Conclusion

Snowflake Time Travel transforms recovery from a painful process into a simple SQL operation.

If you work with production data pipelines, mastering Time Travel is essential.

Because eventually, someone *will* run:

```
DELETE FROM production_table;
```

And when that day comes, Time Travel might save your entire week.